Adaptive Sketching for Open-Leg Tensor Network Contraction

Kaiyu Shen

Abstract

本文研究一类带开放腿的张量网络收缩问题：目标不是计算单个标量结果，而是生成完整的输出张量。在误差评估、可视化与跨系统接口等场景中，下游真正需要的是完整的稠密输出。此时，问题的关键不在于改变输出形式，而在于如何在保持完整输出要求不变的前提下，降低生成该输出所需的内部收缩代价。为此，本文提出 AS-TNC 方法：先对输出索引空间做分块，再在交互图上构造划分树，并沿树递归维护分隔状态；方法始终保持输出边界显式存在，只对内部接口做自适应低秩近似，并在内部合并处通过基于算子访问的隐式草图合并避免显式形成膨胀接口。

实验围绕多种经典张量网络展开，主要关注三个问题：是否存在可用的精度–时间折中、收益是否确实来自收缩阶段，以及近似强度能否稳定控制误差。结果表明，在代表性实验中，AS-TNC 将相对误差控制在 4.81×10^{-3} 以内，并相对直接 TNC 获得 $13.98\times$ 到 $84.05\times$ 的加速；相比之下，固定小秩基线虽然更快，但相对误差达到 0.65–0.98。进一步分析表明，加速几乎完全来自收缩阶段；机制分析进一步说明，隐式草图合并是主要效率来源，输出分块也具有实质作用。近似强度扫描实验则表明，近似强度可以有效控制误差与时间的权衡。

1 引言

张量网络为高维数组的表示与计算提供了紧凑而有结构的形式，已经广泛用于量子多体计算、统计物理、图模型推断以及张量算法等方向 [19]。围绕张量网络收缩（TNC），已有工作主要关心三类问题：如何设计更好的收缩顺序，如何控制中间状态规模，以及如何通过中间状态压缩来降低近似收缩的代价 [18, 7, 15, 11, 14, 20, 10, 17]。现有研究大多围绕标量结果计算、局部可观测量计算或概率推断中的局部查询展开，而非显式生成保留开放腿后的完整输出张量 [19, 4, 20, 13]。相比之下，一旦要求完整物化网络，计算与存储开销会显著膨胀 [15]。本文关注的正是这一类带开放腿张量网络的完整输出计算问题。

这一问题与通常的近似收缩有一个根本差别：当完整输出本身就是任务目标时，算法不能将其替换为压缩表示，也不能仅返回少量局部结果。因此，我们关注的问题不是改变输出形式，而是在保持完整输出要求不变的前提下，降低生成这一输出的计算开销。若把端到端时间写成

$$T_{\text{total}} = T_{\text{contract}} + T_{\text{emit}},$$

我们将 T_{contract} 理解为生成目标输出之前的内部计算开销，即网络内部的收缩、合并及相关中间状态处理；将 T_{emit} 理解为把这些结果显式物化为完整输出张量的开销。由于完整输出本身必须被生成，T_{emit} 往往不可回避，因此方法设计的主要优化对象通常是 T_{contract}。

基于这一观察，本文研究一种面向完整物化的近似收缩框架。基本思路是：在网络交互图上构造一棵划分树，并沿树递归维护各个子网络的分隔状态；在整个过程中，输出边界始终保持显式，只对内部传播的分隔状态做近似。于是，本文近似的不是最终输出对象，而是“为了得到该输出而必须经过的内部中间状态”。这也是本文与直接压缩输出端的方法，以及以标量收缩为目标的方法之间最本质的区别。

从表示上看，每个子树状态都可以理解为一个从输出侧到边界侧的接口矩阵；本文用低秩因子化在树上递归传播这一状态，并在每次合并后重新压缩，以抑制中间维度增长。进一步地，针对内部节点合并时容易出现的维度膨胀，本文引入基于算子访问的隐式草图合并，不显式形成完整的合并接口，而是直接恢复其主导子空间。因此，本文的核心并不是某一个孤立的局部技巧，而是把划分树递归、分隔状态近似与草图驱动的合并压缩组织成一条统一的完整物化流程。在这条流程里，输出语义保持不变，近似只服务于降低内部收缩成本。

实验部分围绕三个问题展开：第一，AS-TNC 在 TNC 任务中能否形成清楚而可用的精度–时间折中；第二，这些收益是否确实来自收缩阶段，以及哪些机制在较难网络上最关键；第三，近似强度是否能稳定控制误差。结果表明，在代表性经典网络上，AS-TNC 将相对误差控制在 4.81×10^{-3} 以内，同时相对直接 TNC 获得 $13.98\times$ 到 $84.05\times$ 的加速；而固定秩基线采用固定小秩，虽然更快，却带来 0.65 到 0.98 的大误差。进一步分析表明，端到端收益几乎完全来自收缩阶段；隐式草图合并是稳定的效率来源，输出分块也具有实质作用。近似强度扫描实验则显示，近似强度是控制误差的旋钮。

本文的贡献可以概括为三点：

- 针对带开放腿张量网络的完整稠密物化任务，本文将近似的位置从输出端移到内部收缩过程，关注在保持完整输出要求不变的前提下降低内部计算代价。
- 本文提出 AS-TNC 框架：在划分树上递归维护分隔状态，通过自适应低秩压缩控制状态规模，并在内部合并处引入隐式草图合并，以避免显式形成膨胀接口。
- 本文在 Ring-8、Tree-8、Grid 3×3 与 Random-8 上系统验证了该方法，结果表明 AS-TNC 能够形成清楚的精度–时间折中，并揭示了其主要收益来源与关键机制。

2 相关工作

精确张量网络收缩。 精确张量网络收缩的经典研究主要围绕收缩顺序、收缩树、树宽以及切片策略展开。这一路线的核心结论很明确：真正决定复杂度的往往不是单个张量本身，而是中间状态的规模，以及由此诱导出的时间与空间开销 [18, 7, 15, 11]。在此基础上，index slicing 又进一步说明，当一次计算难以承受时，可以把部分自由度切出来，转化为许多更小的子任务来并行执行或分批完成 [14]。这些工作为本文提供了两个直接启发：一是困难首先来自中间分隔结构的膨胀；二是按块组织计算可以成为完整物化中的重要系统手段。不过，这一方向的主要目标仍然是精确收缩，而不是在完整输出仍需显式生成的前提下进行近似优化。

近似收缩与中间状态压缩。 当精确收缩代价过高时，近似张量网络方法通常通过压缩中间对象来控制复杂度。在规则拓扑上，TRG 等工作已经表明：面对不可承受的中间状态，能否做出有效截断，是近似收缩能否成立的关键 [16]。近年来，这一思路逐渐从规则晶格走向一般图结构。Pan 等人提出了适用于任意连通结构的一般性近似收缩算法 [20]；Gray 和 Chan 将近似直接纳入收缩规划过程，使压缩与收缩顺序能够联合优化 [10]；Ma 等人则发展了面向一般图的 density-matrix 风格近似框架 [17]。这些工作说明，一般图上的近似收缩已经可以系统地开展。相比已有面向一般图的近似收缩方法，本文关注的是带开放腿张量网络的完整物化任务。由于输出张量需要被完整生成，算法不能改变其形式，因此更合理的做法是在保持输出边界不变的前提下，对网络内部传播的分隔状态进行近似处理。

随机化低秩近似与草图。 本文的方法也借鉴了随机化数值线性代数与草图的思想。随机化低秩近似表明，在许多大规模问题中，可以先用随机投影快速捕捉主导子空间，再在较小的子空间内做稳定分解，而不必一开始就构造昂贵的精确分解 [12]。更一般地，Count Sketch 与 Tensor Sketch 等方法展示了随机线性映射在高维对象压缩中的作用 [2, 21]。非常近期的工作也开始直接讨论张量网络收缩中的草图近似 [13]。在本文中，随机化压缩是分隔状态框架中的实现工具之一，其作用是降低内部状态压缩的代价，而不是改变本文的问题设定。

与图模型消去方法的关系。 从算法结构上看，本文也与图模型中的 bucket elimination 和 mini-bucket 有相通之处 [3, 5]。这些方法同样强调：真正向上层传播的不是局部对象的全部细节，而是与外部仍然相关的接口信息；当接口过大时，就需要在中间层面做受控近似。本文将这一思想放在带开放腿张量网络的完整物化任务下重新组织，并把输出边界保持视为方法定义的一部分。

3 方法

3.1 整体思路

设 $\mathcal{N}$ 为一个带开放腿的张量网络，其开放标签集合记为 $\mathcal{O}$，内部收缩标签集合记为 $\mathcal{I}$。将所有内部标签收缩后，可得到完整输出张量

$$X = \mathrm{Mat}(\mathcal{N}) \in \mathbb{R}^{d_1 \times \cdots \times d_{|\mathcal{O}|}}.$$

其中 Mat 表示将网络物化为完整输出张量。相应地，端到端时间可写为

$$T_{\text{total}} = T_{\text{contract}} + T_{\text{emit}},$$

其中 T_{contract} 表示内部收缩与近似本身的代价，T_{emit} 表示将结果写回最终输出缓冲区的代价。在完整物化场景下，T_{emit} 通常难以避免，因此真正值得优化的主要是 T_{contract}。基于这一判断，本文把近似严格限制在内部传播的分隔状态上，而不直接压缩输出边界。

整条方法主线可以概括为三步：

1. 在网络交互图上构造一棵二叉划分树，把一般图上的收缩过程写成一系列二叉合并；
2. 对每个子树，用分隔状态来概括它对外仍然可见的信息；
3. 沿树递归合并这些分隔状态，并在每次合并后重新压缩，以控制中间状态规模。

3.2 划分树与分隔状态

为了统一组织一般图上的递归收缩，我们在张量网络的交互图上构造一棵划分树（partition tree）。树上的每个节点 v 对应一个子网络 $\mathcal{N}_v$；叶节点对应单个原始张量，内部节点对应两个子网络的合并，根节点对应整个网络。

对任意节点 v，我们关心三类标签：

- 输出标签 $\mathcal{O}_v$：该子树最终仍通向输出侧的标签；
- 边界标签 $\mathcal{B}_v$：该子树与外部其余网络连接所需保留的标签；
- 切割标签 $\mathcal{C}_v$：当 v 由两个孩子合并而成时，连接左右孩子并在本层被消去的标签。

根节点不再与外部网络相连，因此其边界标签为空；叶节点不再继续拆分，因此没有切割标签。

给定输出块 Ω_b 后，我们将节点 v 的分隔状态定义为

$$M_v \in \mathbb{R}^{d_{\text{open}}(v) \times d_{\text{bnd}}(v)},$$

其中

$$d_{\text{open}}(v) = \prod_{\ell \in \mathcal{O}_v} d_\ell, \qquad d_{\text{bnd}}(v) = \prod_{\ell \in \mathcal{B}_v} d_\ell.$$

这里 d_ℓ 表示标签 ℓ 在当前输出块中的局部维度；若该标签未被当前块限制，则 $d_\ell = d_\ell$。矩阵 M_v 的行对应输出侧配置，列对应边界侧配置，因此它刻画了当前子树在输出块 Ω_b 下对父节点可见的局部行为。

直接显式存储 M_v 往往代价过高，因此我们采用低秩因子形式

$$M_v \approx A_v\big(B_v\big)^\top,$$

其中

$$A_v \in \mathbb{R}^{d_{\text{open}}(v)\times r_v}, \qquad B_v \in \mathbb{R}^{d_{\text{bnd}}(v)\times r_v},$$

r_v 是节点 v 的接口秩。这种写法把输出侧与边界侧显式分开，也清楚地说明了本文的近似位置：输出边界保持不变，近似只发生在分隔状态上。

3.3 树上的递归合并

在定义了分隔状态之后，整棵划分树上的计算就可以写成统一的递归。

叶节点。 对叶节点 v，其子网络只包含一个原始张量。在给定输出块后，我们先将与当前块相关的开放标签限制到局部索引范围，再把张量重排成

$$T_v \in \mathbb{R}^{d_{\text{open}}(v)\times d_{\text{bnd}}(v)}.$$

若不做近似，则 T_v 就是该叶节点的精确分隔状态；若做近似，则对 T_v 做低秩压缩，得到

$$M_v \approx A_v\big(B_v\big)^\top.$$

内部节点。 设内部节点 v 的两个孩子为 v_L 和 v_R，其分隔状态分别为

$$M_{v_L} \approx A_{v_L}\big(B_{v_L}\big)^\top, \qquad M_{v_R} \approx A_{v_R}\big(B_{v_R}\big)^\top.$$

父节点的构造可分为三步。

(i) 合并输出侧。 父节点的输出标签满足

$$\mathcal{O}_v = \mathcal{O}_{v_L} \cup \mathcal{O}_{v_R}, \qquad \mathcal{O}_{v_L} \cap \mathcal{O}_{v_R} = \varnothing.$$

因此，父节点的输出侧配置空间是左右孩子输出侧配置空间的笛卡尔积。在因子层面，这一步对应于将两个孩子的输出侧因子做外积，等价地可写成 Kronecker 积

$$A_{\text{mat}} = A_{v_L} \otimes A_{v_R},$$

其列数为 $r_{v_L} r_{v_R}$。

(ii) 收缩切割标签，保留边界标签。 设父节点保留的边界标签集合为 $\mathcal{B}_v$，本层被消去的切割标签集合为 $\mathcal{C}_v$。则父节点未压缩的边界侧因子可以看成对两个孩子的边界侧因子沿 $\mathcal{C}_v$ 做收缩：

$$B_{\text{mat}} = B_{v_L} \times_{\mathcal{C}_v} B_{v_R}.$$

直观地看，这一步完成了“该消去的内部边被消去，该保留给父节点的边界仍然保留”。

(iii) 重新压缩。 经过前两步后，父节点得到未压缩表示

$$\widetilde{M}_v = A_{\text{mat}} B_{\text{mat}}^\top.$$

由于其中的中间列数已经膨胀到 $r_{v_L} r_{v_R}$，若不立即重压缩，秩会在树上迅速增长。因此，我们对 $\widetilde{M}_v$ 再做一次低秩压缩，写回统一的分隔状态形式

$$M_v \approx A_v\big(B_v\big)^\top.$$

3.4 自适应秩选择

在分隔状态递归中，每次叶节点构造和内部节点合并之后，都要对当前分隔状态重新压缩，以防止秩在树上不断膨胀。最直接的做法是为所有节点统一指定一个固定秩 r。这种方案实现简单，但它默认划分树上不同节点的分隔状态难度大致相同；这一假设在实际中通常并不成立。

不同节点对应的子树规模、边界结构、当前输出块下的局部自由度以及谱衰减形状都可能差别很大。有些节点本身较简单，较低秩已经足够；另一些节点则包含更复杂的内部耦合，若仍采用统一秩预算，就会过早丢失关键信息。因此，固定秩往往会同时带来两类问题：在简单节点上保留了多余通道，在困难节点上又压得不够。

基于这一观察，本文在每个待压缩节点上按局部误差容忍度自适应选择秩，而不是预先指定统一目标秩。设某个待压缩接口矩阵为

$$M \in \mathbb{R}^{m\times n},$$

其奇异值分解为

$$M = U\Sigma V^\top, \qquad \Sigma = \mathsf{diag}(\sigma_1, \sigma_2, \ldots).$$

若取其秩-r 截断近似

$$M_r = U_r \Sigma_r V_r^\top,$$

则 Frobenius 范数下的相对误差为

$$\frac{\|M - M_r\|_F}{\|M\|_F} = \left(\frac{\sum_{i>r}\sigma_i^2}{\sum_i \sigma_i^2}\right)^{1/2}.$$

因此，给定容忍度 τ，我们选择满足

$$\left(\frac{\sum_{i>r}\sigma_i^2}{\sum_i \sigma_i^2}\right)^{1/2} \le \tau$$

的最小秩 r。在实现中，叶节点与内部合并节点可分别使用不同的容忍度，记为 τ_{leaf} 与 τ_{merge}。

这种写法的好处是，它把“每个节点保留多少通道”转化成“每个节点允许损失多少相对能量”。谱衰减快的简单节点会自动停在较低秩，而谱衰减慢的困难节点会自动保留更多通道。因此，自适应秩选择的关键不在于简单增大平均秩，而在于按局部难度重新分配秩预算。

3.5 隐式草图合并

仅靠自适应秩选择还不够。在内部节点处，若左右孩子的秩分别为 r_L 和 r_R，则父节点在重压缩之前自然会出

现一个秩乘积 $r_L r_R$。如果先显式构造合并后的大接口矩阵，再对它做普通低秩分解，就会遇到“先膨胀，再压回去”的中间瓶颈。为此，本文在合并节点上引入隐式草图合并：不显式形成完整的合并接口，而是通过算子访问直接恢复其主导子空间 [12, 13]。

设合并后待压缩的父节点接口为

$$\widetilde{M} \in \mathbb{R}^{m\times n}.$$

在显式路径中，它可以写成

$$\widetilde{M} = A_{\text{mat}} B_{\text{mat}}^{\top},$$

其中 A_{mat} 来自输出侧因子的组合，B_{mat} 来自边界侧因子沿切割标签的收缩。随机化压缩真正需要的不是 $\widetilde{M}$ 的全部条目，而是对线性算子

$$x \mapsto \widetilde{M}x, \qquad y \mapsto \widetilde{M}^{\top} y$$

的访问。在本文的合并结构下，这两个乘法可写为

$$\widetilde{M}x = A_{\text{mat}}\big(B_{\text{mat}}^{\top}x\big), \qquad \widetilde{M}^{\top}y = B_{\text{mat}}\big(A_{\text{mat}}^{\top}y\big).$$

因此，整个压缩过程都可以建立在算子应用之上，而不必先显式形成完整的 $\widetilde{M}$。

在此基础上，我们采用 randomized SVD 的标准流程 [12]。首先生成高斯随机测试矩阵

$$\Omega \in \mathbb{R}^{n\times \ell}, \qquad \Omega_{ij} \sim \mathcal{N}(0,1),$$

其中 $\ell = k + p$，k 是目标秩的估计规模，p 是 oversampling 参数。然后构造样本矩阵

$$Y_0 = \widetilde{M}\Omega.$$

这里的 Y_0 并不是通过显式矩阵乘法得到，而是通过对 Ω 逐列或成块应用算子 $\widetilde{M}$ 得到。当谱衰减较慢时，为了更稳定地提取主导子空间，还可以做 q 次幂迭代：

$$Y_{t+1} = \widetilde{M}\big(\widetilde{M}^{\top}Y_t\big), \qquad t = 0, 1, \ldots, q-1.$$

随后对最终样本矩阵做正交化，得到

$$Q = \operatorname{orth}(Y_q) \in \mathbb{R}^{m\times \ell},$$

其中 Q 近似张成了 $\widetilde{M}$ 的主导列空间。再将原问题压缩到小矩阵

$$M_{\text{proj}} = Q^{\top}\widetilde{M} \in \mathbb{R}^{\ell\times n}$$

上，并对其做精确 SVD：

$$M_{\text{proj}} = \widetilde{U}\Sigma V^{\top}.$$

到这一步为止，原来的大接口已经被转化成一个小矩阵上的谱截断问题。直接采用固定秩可以直接保留前 r 个奇异方向；而本文主线采用基于尾能量的自适应截断：给定合并容忍度 τ，选择满足

$$\left(\frac{\sum_{i>r}\sigma_i^2}{\sum_i \sigma_i^2}\right)^{1/2} \le \tau$$

的最小秩 r。设据此得到的截断结果为

$$\widetilde{M} \approx Q\widetilde{U}_r \Sigma_r V_r^{\top},$$

为了与全文统一的分隔状态表示保持一致，我们将其写回为因子形式

$$A = (Q\widetilde{U}_r)\Sigma_r^{1/2}, \qquad B = V_r \Sigma_r^{1/2},$$

从而得到

$$\widetilde{M} \approx AB^{\top}.$$

这里，自适应秩选择决定“最终保留多少通道”，而随机化压缩决定“如何在不显式收缩大接口的情况下找到这些通道”。

3.6 算法总览

为了便于整体把握，Algorithm 1 给出本文方法的主流程。它把前文的几个核心部件统一起来：输出分块负责组织完整物化过程，划分树负责把一般图上的收缩写成递归二叉合并，分隔状态负责在树上向上传播局部信息，而重压缩则用于控制中间状态规模。划分树构造、局部块键缓存、精确收缩基线的实现接口以及随机化压缩的可重复性设置见附录 Section A.1。

4 实验

本节围绕四个问题组织实验：第一，AS-TNC 在带开放腿张量网络的完整物化任务中，是否形成了清楚而可用的精度–时间折中；第二，这些收益是否确实来自内部收缩，而不是输出写回；第三，隐式草图合并、缓存复用与输出分块分别起什么作用；第四，近似强度是否能够作为稳定的误差调控旋钮。

4.1 实验设置

我们主要报告四类有代表性的经典网络：Ring-8 [1]、Tree-8 [6]、Grid 3×3 [16, 19] 与 Random-8 [8, 9]；其中，8 表示节点数，3×3 表示二维网格规模。每个节点都带 1 条开放腿，物理维统一取 3，键维统一取 4。除特别说明外，AS-TNC 指默认 ASTNC-L2 配置；其在主实验中的默认物化设置为：启用输出分块，取 `block_labels=2`、`chunk_size=1`，同时启用隐式草图合并与块间缓存复用。对比方法包括精确收缩基线 exact-TNC、固定秩基线与 AS-TNC。固定秩基线统一设置秩为 4，目的是提供一个明确的小秩快速近似参照。其他细节（网络构造、容忍度、分块策略与计时口径）见附录 Section A.2。

4.2 主结果

Table 1 给出了四个典型网络上的主结果。可以看到，固定秩基线虽然很快，但在四个网络上都产生了明显失真：其相对误差介于 0.65 到 0.98 之间。相比之下，默认 AS-TNC 配置在四个网络上都将相对误差控制在 4.81×10^{-3} 以内，同时相对 exact-TNC 仍获得了 8.97× 到 88.45× 的加速。最具区分度的结果来自 Grid 3×3：exact-TNC

算法 1 AS-TNC 的完整物化主流程

输入： 张量网络 $\mathcal{N}$，输出块集合 $\{\Omega_b\}_{b=1}^{B}$，压缩配置 θ
输出： 完整输出张量 X

在 $\mathcal{N}$ 的交互图上构造划分树 $\mathcal{T}$
初始化输出缓冲区 X
初始化状态缓存 $\mathcal{C}$
for 每个输出块 Ω_b **do**
  $S_{\text{root}} \leftarrow$ 构造状态$(\text{root}(\mathcal{T}), \Omega_b, \theta, \mathcal{C})$
  $X[\Omega_b] \leftarrow$ 根节点读出(S_{root})
end for
return X

function 构造状态 $(v, \Omega_b, \theta, \mathcal{C})$
  if (v, Ω_b, θ) 对应状态已在缓存中可复用 **then**
    return 缓存状态
  end if
  if v 是叶节点 **then**
    在输出块 Ω_b 的限制下形成局部张量 T_v
    在 $(\mathcal{O}_v, \mathcal{B}_v)$ 上构造分隔状态 M_v
    视需要对 M_v 做压缩
    将状态存入缓存并返回
  else
    $S_L \leftarrow$ 构造状态$(v_L, \Omega_b, \theta, \mathcal{C})$
    $S_R \leftarrow$ 构造状态$(v_R, \Omega_b, \theta, \mathcal{C})$
    合并 S_L 与 S_R 的输出侧因子
    沿切割标签 $\mathcal{C}_v$ 收缩边界侧因子
    对合并后的分隔状态重新压缩
    将状态存入缓存并返回
  end if
end function

function 根节点读出 (S_{root})
  收回根状态的最后一条秩维
  return 与 Ω_b 对应的稠密输出块
end function

需要 47.25 秒，而 AS-TNC 将总时间降到 0.5342 秒，相对误差仅为 4.08×10^{-3}。在四个主结果网络上，AS-TNC 的几何平均加速达到 35.08×，平均相对误差为 2.68×10^{-3}。这表明 AS-TNC 在保持较高精度的同时，能够显著降低完整物化所需时间。

4.3 收缩主导与机制作用

首先，我们考察端到端收益究竟来自哪里。由 Table 2 可见，在四个主结果网络上，默认 AS-TNC 配置的平均 $t_{\text{contract}}/t_{\text{total}}$ 达到 0.9993；其平均收缩加速比与总加速比分别为 46.25× 与 46.23×，几乎完全一致。这说明 AS-TNC 的主要收益确实来自内部收缩阶段，而不是输出写回。以 Grid 3×3 为例，收缩加速为 88.48×，总加速为 88.45×，二者在数值上几乎重合。

接着，我们在 Grid 3×3 这一最具区分度的网络上隔离关键机制。结果如 Table 3 所示。去掉隐式草图合并后，误差几乎不变，但总时间恶化到默认设置的 2.21×；这说明隐式草图合并是稳定且核心的效率来源。去掉缓存复用后，误差同样几乎不变，但总时间上升为默认设置的 1.23×，表明块间状态复用能够带来额外但稳定的系统收益。相比之下，去掉输出分块后，总时间并未显著变差，但误差从 4.08×10^{-3} 上升到 1.06×10^{-2}，同时内部峰值秩由 64 上升到 222。这说明输出分块在当前小规模设置下更像是在塑造一个更优的精度–效率 operating point，而不是一个单调的纯加速开关。

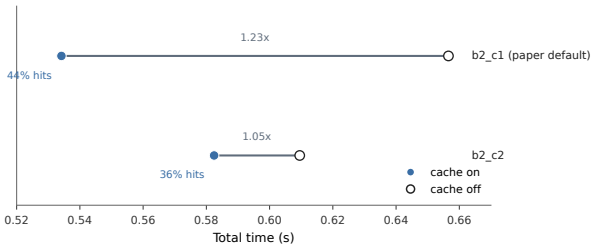

Figure 1: Grid 3×3 上 cache reuse 的成对对照。对齐设置下，开启 cache 在 $b2_c1$（正文默认配置）和 $b2_c2$ 上都带来稳定降时。

为了把缓存复用的收益更直观地呈现出来，我们进一步给出一张成对对照图。由 Figure 1 可见，在对齐设置下，cache 在 $b2_c1$（正文默认配置）与 $b2_c2$ 两组设置上都带来稳定的降时效果；其中默认配置 $b2_c1$ 的提升最明显。这种 paired comparison 比单独报一个命中率数字更直观，也和上面的机制表形成了互补。更完整的 blockwise 设计空间与机制指纹可视化见附录 Figures 3 and 4。

4.4 近似强度与连续前沿

我们接着考察近似强度是否能够作为稳定的误差调控旋钮。Table 4 给出了离散的三档强度扫描结果。可以看到，从 L1 到 L2 再到 L3，误差整体上呈现清楚的上升趋势。以 Grid 3×3 为例，相对误差从近乎机器精度上升到 4.08×10^{-3}，再增至 1.81×10^{-2}；Tree-8 上则从近乎零误差上升到 4.81×10^{-3}，再达到 4.86×10^{-2}。四个网络上的平均相对误差分别为近乎零、3.59×10^{-3} 与 2.37×10^{-2}。这说明近似强度首先是一个有效的误差调控旋钮：更弱的近似更接近精确结果，而更强的近似则带来更大的误差。

除了离散的三档设置，我们还做了连续容忍度扫描。结果表明，AS-TNC 并不是只有 L1/L2/L3 三个孤立点，而是形成了一条可调的精度–时间前沿。由 Figure 2 可见，Grid 3×3 呈现出更宽的速度–精度折中区域：在更严格的设置下，方法可以在近乎零误差时仍保持接近 100× 的加速；而随着容忍度继续放宽，误差逐渐上升到 10^{-2} 量级。相比之下，Random-8 上的 sweep 更为稳定，说明较容易的实例并不会像 Grid 3×3 那样呈现显著的 trade-off 展开。需要说明的是，这条前沿并非严格单调，因此更合适的解读方式是从中选取 non-dominated operating points，而不是把它看成一条简单的单调曲线。

5 结论

本文研究了带开放腿张量网络的完整稠密物化问题。与传统近似收缩主要面向标量结果或局部查询不同，本文关注的是这样一类任务：最终结果必须以完整张量的形式被显式生成，因此方法设计不能通过改变输出形式来规避计

Table 1: 四个典型网络上的主结果：exact-TNC、固定秩基线与 AS-TNC 的精度–时间折中对比。

Case	exact-TNC time (s)	fixed-rank rel. error	fixed-rank speedup	AS-TNC rel. error	AS-TNC speedup
Ring-8	3.3502	0.8058	112.70×	1.83×10^{-3}	41.61×
Tree-8	1.0964	0.6524	59.53×	4.81×10^{-3}	45.88×
Grid 3×3	47.2469	0.9789	444.01×	4.08×10^{-3}	88.45×
Random-8	0.7339	0.9283	31.53×	$<10^{-12}$	8.97×
Summary	—	—	—	avg. $=2.68\times10^{-3}$	gmean $=35.08\times$

Table 2: 时间阶段分解证据：AS-TNC 的端到端收益几乎全部来自收缩阶段，而非输出写回。

Case	$t_{\text{contract}}/t_{\text{total}}$	$t_{\text{emit}}/t_{\text{total}}$	contraction speedup	total speedup
Ring-8	0.9995	0.000470	41.63×	41.61×
Tree-8	0.9986	0.001342	45.94×	45.88×
Grid 3×3	0.9997	0.000303	88.48×	88.45×
Random-8	0.9994	0.000480	8.97×	8.97×
Average	0.9993	0.000649	46.25×	46.23×

Table 3: Grid 3×3 上的核心机制证据。默认配置对应 paper default：`block_labels=2`、`chunk_size=1`、cache on、implicit sketch on。

Setting	rel. error	total time (s)	time ratio vs. default	cache hit rate
AS-TNC (default)	4.08×10^{-3}	0.5342	1.00	0.440
no-blockwise	1.06×10^{-2}	0.5265	0.986	0
no-implicit	4.08×10^{-3}	1.1792	2.21	0.440
no-cache	4.08×10^{-3}	0.6566	1.23	0

算代价。基于这一设定，本文将优化重点放在内部收缩过程，并提出了 AS-TNC 框架。

围绕这一目标，本文将完整物化过程组织为一条统一流程：先对输出索引空间进行分块，再在网络交互图上构造划分树，并沿树递归维护保持输出边界显式存在的分隔状态；在状态传播过程中，通过自适应低秩压缩和隐式草图合并抑制内部接口在合并阶段的维度膨胀。因而，AS-TNC 近似的不是最终输出对象，而是生成该输出过程中必须经过的内部分隔状态。

实验结果表明，AS-TNC 在 Ring-8、Tree-8、Grid 3×3 与 Random-8 上能够形成清楚且可用的精度–时间折中：在保持较小相对误差的同时，相对直接 TNC 获得显著加速。进一步的阶段拆解与机制分析说明，端到端收益几乎完全来自收缩阶段；其中，隐式草图合并是更核心也更稳定的效率来源，而输出分块在较困难的网络上同样具有实质作用。强度扫描实验则进一步表明，近似强度可以作为控制误差与时间权衡的有效调节旋钮。

总体而言，本文说明了：当完整输出是不可改变的任务要求时，近似收缩的合理位置并不在输出端，而在内部收缩过程中传播的分隔状态。沿着这一思路，划分树递归、分隔状态表示与草图驱动压缩可以被组织成一个统一且有效的完整物化框架。未来工作可进一步面向更大规模和更复杂的任务，继续完善选择性近似、缓存复用和更丰富的草图策略，使这种面向完整物化的近似收缩方法在更广泛场景下发挥作用。

Table 4: 近似强度调控证据：从 L1 到 L2 再到 L3，误差整体上升。

Case	L1 rel. error	L2 rel. error	L3 rel. error
Ring-8	$<10^{-12}$	1.83×10^{-3}	1.55×10^{-2}
Tree-8	$<10^{-12}$	4.81×10^{-3}	4.86×10^{-2}
Grid 3×3	$<10^{-12}$	4.08×10^{-3}	1.81×10^{-2}
Random-8	$<10^{-12}$	$<10^{-12}$	1.26×10^{-2}
Average rel. error	$<10^{-12}$	3.59×10^{-3}	2.37×10^{-2}

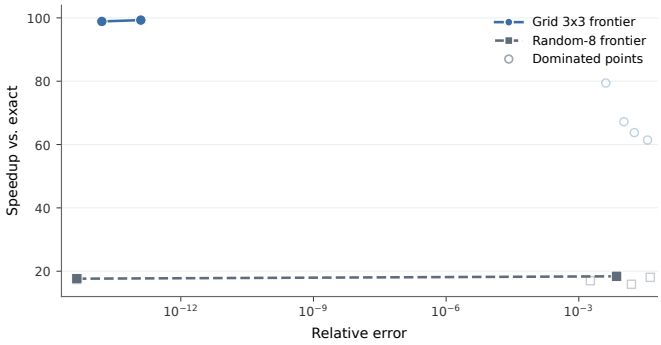

Figure 2: 连续容忍度扫描得到的精度–时间前沿。实心标记与连线对应 non-dominated frontier，浅色空心点对应被支配的 sweep setting。Grid 3×3 表现出更宽的 trade-off 区域，而 Random-8 更为稳定。

References

[1] J. A. Bondy and U. S. R. Murty. *Graph Theory*. Springer, 2008.

[2] Moses Charikar, Kevin Chen, and Martin Farach-Colton. "Finding Frequent Items in Data Streams". In: *Theoretical Computer Science* 312.1 (2004), pp. 3–15. doi: 10.1016/S0304-3975(03)00400-6.

[3] Rina Dechter. "Bucket Elimination: A Unifying Framework for Probabilistic Inference". In: *Proceedings of the Twelfth Conference on Uncertainty in Artificial Intelligence*. 1996, pp. 211–219.

[4] Rina Dechter. "Bucket elimination: A unifying framework for probabilistic inference". In: *Learning in graphical models*. Springer, 1998, pp. 75–104.

[5] Rina Dechter and Irina Rish. "Mini-Buckets: A General Scheme for Generating Approximations in Automated Reasoning". In: *Proceedings of the Fifteenth International Joint Conference on Artificial Intelligence*. 1997, pp. 1297–1303.

[6] Reinhard Diestel. *Graph Theory*. 5th ed. Springer, 2017.

[7] Eugene F. Dumitrescu, Allison L. Fisher, Timothy D. Goodrich, Travis S. Humble, Blair D. Sullivan, and Andrew L. Wright. "Benchmarking Treewidth as a Practical Component of Tensor Network Simulations". In: *PLOS ONE* 13.12 (2018), e0207827. doi: 10.1371/journal.pone.0207827.

[8] Paul Erdős and Alfréd Rényi. "On Random Graphs I". In: *Publicationes Mathematicae (Debrecen)* 6 (1959), pp. 290–297.

[9] Edgar N. Gilbert. "Random Graphs". In: *The Annals of Mathematical Statistics* 30.4 (1959), pp. 1141–1144. doi: 10.1214/aoms/1177706098.

[10] Johnnie Gray and Garnet Kin-Lic Chan. "Hyperoptimized Approximate Contraction of Tensor Networks with Arbitrary Geometry". In: *Physical Review X* 14.1 (2024), p. 011009. doi: 10.1103/PhysRevX.14.011009.

[11] Johnnie Gray and Stefanos Kourtis. "Hyperoptimized Tensor Network Contraction". In: *Quantum* 5 (2021), p. 410. doi: 10.22331/Q-2021-03-15-410.

[12] Nathan Halko, Per-Gunnar Martinsson, and Joel A. Tropp. "Finding Structure with Randomness: Probabilistic Algorithms for Constructing Approximate Matrix Decompositions". In: *SIAM Review* 53.2 (2011), pp. 217–288. doi: 10.1137/090771806.

[13] Mike Heddes, Igor Nunes, Tony Givargis, and Alex Nicolau. "Approximating Tensor Network Contraction with Sketches". In: *arXiv preprint arXiv:2603.07387* (2026).

[14] Cupjin Huang, Fang Zhang, Michael Newman, Xiaotong Ni, Dawei Ding, Junjie Cai, Xun Gao, Tenghui Wang, Feng Wu, Gengyan Zhang, et al. "Efficient Parallelization of Tensor Network Contraction for Simulating Quantum Computation". In: *Nature Computational Science* 1 (2021), pp. 578–587. doi: 10.1038/s43588-021-00119-7.

[15] Jonathan Jakes-Schauer, David Anekstein, and Pawel Wocjan. "Carving-width and Contraction Trees for Tensor Networks". In: *arXiv preprint arXiv:1908.11034* (2019).

[16] Michael Levin and Cody P. Nave. "Tensor Renormalization Group Approach to Two-Dimensional Classical Lattice Models". In: *Physical Review Letters* 99.12 (2007), p. 120601. doi: 10.1103/PhysRevLett.99.120601.

[17] Linjian Ma, Matthew T. Fishman, E. Miles Stoudenmire, and Edgar Solomonik. "Approximate Contraction of Arbitrary Tensor Networks with a Flexible and Efficient Density Matrix Algorithm". In: *Quantum* 8 (2024), p. 1580. doi: 10.22331/q-2024-12-27-1580.

[18] Igor L. Markov and Yaoyun Shi. "Simulating Quantum Computation by Contracting Tensor Networks". In: *SIAM Journal on Computing* 38.3 (2008), pp. 963–981. doi: 10.1137/050644756.

[19] Román Orús. "A Practical Introduction to Tensor Networks: Matrix Product States and Projected Entangled Pair States". In: *Annals of Physics* 349 (2014), pp. 117–158. doi: 10.1016/j.aop.2014.06.013.

[20] Feng Pan, Pengfei Zhou, Sujie Li, and Pan Zhang. "Contracting Arbitrary Tensor Networks: General Approximate Algorithm and Applications in Graphical Models and Quantum Circuit Simulations". In: *Physical Review Letters* 125.6 (2020), p. 060503. doi: 10.1103/PhysRevLett.125.060503.

[21] Ninh Pham and Rasmus Pagh. "Fast and Scalable Polynomial Kernels via Explicit Feature Maps". In: *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2013, pp. 239–247. doi: 10.1145/2487575.2487591.

A 附录

A.1 方法细节与机制补充

为了使上述方法具有可复现性，这里进一步说明划分树构造、输出分块、根节点读出、块间缓存以及精确收缩基线的具体实现机制。

划分树构造。 设张量网络的交互图为 $G=(V,E,w)$，其中每个节点对应一个原始张量；若两个张量共享内部标签，则在它们之间连边。当前实现中，边权由共享内部标签的维度贡献累加得到。在任意子图 $G[U]$ 上，我们递归执行一次二分：若子图连通，则使用 Stoer–Wagner min-cut 得到 U_L, U_R；若子图退化或无边，则回退为“单节点 vs 其余节点”的划分。据此自底向上构造一棵二叉划分树。对于任意树节点 v（对应子网络 $\mathcal{N}_v$），其输出标签、边界标签与切割标签分别记为 $\mathcal{O}_v, \mathcal{B}_v, \mathcal{C}_v$。

输出分块。 设全局开放标签顺序为 $\mathcal{O}=(\ell_1,\dots,\ell_m)$。实现中仅对前 $q=$ `block_label_count` 个开放标签做分块。对每个被分块标签 ℓ_i，其索引集合被按长度 `chunk_size` 划分为若干连续片段

$$I_{\ell_i}^{(1)}, I_{\ell_i}^{(2)}, \dots.$$

一个输出块 Ω_b 即为这些局部片段的笛卡尔积。因此，分块后的输出块不只是输出写回时的调度单元，它还会直接改变各子树在当前块下的局部输出维度。

局部块键与块间缓存。 对树节点 v 和输出块 Ω_b，我们定义其局部块键

$$\kappa_v(\Omega_b)=\big\{(\ell, I_{b,\ell}) \,:\, \ell\in\mathcal{O}_v,\ \ell \text{ 在当前块中被切片}\big\}.$$

也就是说，$\kappa_v(\Omega_b)$ 只保留该子树真正“看得见”的局部切片条件，而忽略与该子树无关的其他块信息。于是，节点状态缓存可写为

$$K(v,\Omega_b)=\big(\text{node_key}(v),\ \kappa_v(\Omega_b),\ \text{sig}(\theta)\big),$$

其中 $\text{sig}(\theta)$ 表示与压缩相关的配置签名（如秩选择策略、容忍度、随机化参数和优化策略等）。只要两个全局输出块在节点 v 上诱导出相同的 $\kappa_v(\Omega_b)$，则该节点状态即可直接复用。

根节点读出。 当递归合并到根节点时，由于根节点已对应整个网络，其边界标签为空，即 $\mathcal{B}_{\text{root}}=\varnothing$。因此，根状态不再表示一般的“输出侧 $\times$ 边界侧”接口，而是直接退化为当前输出块的稠密结果。若根状态写为 $M_{\text{root}}\approx A_{\text{root}}B_{\text{root}}^{\top}$，则当前块输出由最后一条秩维收回得到。

精确收缩基线。 本文的精确收缩基线 exact-TNC 与 AS-TNC 共用同一张量网络对象和同一输出标签顺序。对每个输出块 Ω_b，exact-TNC 直接调用全网络精确收缩

$$X_{\Omega_b}=\text{contract_full}(\mathcal{N},\Omega_b),$$

其底层实现基于 `opt_einsum` 的精确张量收缩接口，并按给定的 `optimize` 策略执行。因此，AS-TNC 与 exact-TNC 的主要差异不在输出接口，而在于 AS-TNC 以分隔状态递归替代了内部全量收缩过程。

复现说明。 在随机化压缩中，当前实现采用由基础随机种子与状态键共同确定的确定性伪随机数生成方式。因此，同一网络、同一输出块、同一树节点和同一配置下，得到的随机化压缩结果是可重复的。

A.2 关键实验配置与可复现细节

本小节补充正文未展开但会显著影响结果解释与复现实验的关键设置。

网络构造规则 主结果使用 Ring-8、Tree-8、Grid 3×3 与 Random-8。其中，Tree-8 使用随机标号树生成（`networkx.random_labeled_tree`），节点数固定为 8；Random-8 使用 Erdős–Rényi 随机图并重复采样直至连通（`networkx.erdos_renyi_graph` + connectedness check），节点数固定为 8，边概率 $p=0.35$。两者都由全局随机种子驱动（主实验种子为 $\{0,1,2\}$）。

容忍度与强度预设 实现中对叶子和合并操作使用不同的容忍度 τ_{leaf} 和 τ_{merge}，强度扫描三档分别为

$$\begin{aligned}
&\text{L1} \quad (5.0\times10^{-4},\ 2.0\times10^{-3})\\
&\text{L2} \quad (1.0\times10^{-3},\ 5.0\times10^{-3})\\
&\text{L3} \quad (3.0\times10^{-3},\ 1.5\times10^{-2})
\end{aligned}$$

另外，AS-TNC 采用 `depth_open` 调度：节点局部容忍度按

$$\tau_{\text{local}}=\tau_{\text{base}}\cdot(1.5)^{\text{depth}}/(\max(1,\text{open_count}))^{0.5}$$

自适应调整。

输出分块策略 主实验统一启用输出分块策略：`block_labels=2`，`chunk_size=1`。即从开放腿中选 2 个标签作为分块轴，每个轴按单元素切片，逐块收缩并写回完整输出张量。

计时口径与合理性 本文采用同任务、同分块、分阶段的统一计时口径。对每个网络实例，先生成同一组输出分块，然后在该分块上分别运行 exact-TNC 与近似方法；因此不同方法面对的是相同的输出组织与写回负担。计时使用高精度时钟，并将每个输出块的时间拆分为收缩与写回两段并累计，最终报告

$$T_{\text{total}}=T_{\text{contract}}+T_{\text{emit}},$$

同时给出 $t_{\text{contract}}/t_{\text{total}}$ 作为阶段占比指标。速度提升统一定义为

$$\text{speedup}=T_{\text{exact-TNC}}/T_{\text{method}},$$

并在多个随机种子（$\{0,1,2\}$）上汇总，从而降低单次运行抖动对结论的影响。

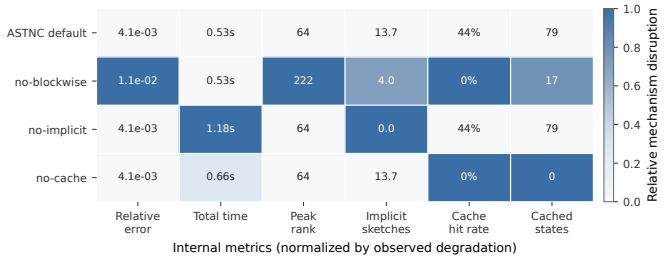

Figure 3: Grid 3×3 上的机制指纹图。按指标归一化后，不同变体的主要退化方向变得更清晰：no-blockwise 主要抬高误差与峰值秩，no-implicit 主要恶化运行时间，no-cache 则主要击穿复用相关统计。

A.3 补充实验结果与机制证据

本节补充正文未完全展开的三个问题：第一，正文默认 AS-TNC 配置在 Grid 3×3 上到底对应什么；第二，不同机制变体的影响究竟落在哪些内部指标上；第三，输出分块为何更像是在定义一个设计空间，而不只是一个简单开关。

正文默认配置的精确定义。　在补强实验中，正文所引用的 Grid 3×3 默认配置对应同一批 core runs：方法为 ASTNC-L2，启用输出分块，取 `block_labels=2`、`chunk_size=1`，同时启用隐式草图合并与块间缓存复用。该配置在三个随机种子上的平均结果为：相对误差 4.08×10^{-3}，收缩时间 0.5340 秒，总时间 0.5342 秒，相对 exact-TNC 的加速为 88.45×，缓存命中率为 0.440。因此，正文中的主结果表、机制表以及后文的 blockwise / cache 补充证据都以这一默认配置为统一参照。

机制指纹视角。　仅看总时间还不足以区分不同机制到底影响了什么。Figure 3 将 Grid 3×3 上的关键内部指标做了归一化汇总。可以看到，去掉隐式草图合并主要伤害运行时间；去掉缓存主要击穿复用相关统计并带来次级降时损失；而去掉输出分块则主要抬高误差与峰值秩。因此，这几个组件并不是在做同一件事，而是在分别塑造精度、运行时间与中间状态规模。

为了保留精确数值，Table 5 进一步给出相同实验的原始统计。可以看到，去掉隐式草图合并后，误差几乎不变，但总时间从 0.5342 秒上升到 1.1792 秒，约恶化为 2.21×；同时隐式草图合并次数由 13.67 下降到 0，说明隐式草图合并确实承担了主要的加速作用。去掉缓存后，误差仍基本不变，但总时间增至 0.6566 秒，表明缓存复用带来了稳定的额外系统收益。相比之下，去掉输出分块后，总时间略降到 0.5265 秒，但误差升高到 1.06×10^{-2}，同时峰值秩从 64 增至 222。这说明在当前小规模设定下，输出分块的价值并不简单体现为单调加速，而是体现在更优的精度–效率折中与更受控的内部状态规模上。

输出分块作为设计空间。　为了进一步理解默认配置为何选择 $b2_c1$（即 `block_labels=2`、`chunk_size=1`），我们在 Grid 3×3 上做了局部 blockwise 扫描。Figure 4 以设计空间的方式展示了这些结果：主图聚焦 practical operating region，而 near-exact 的 $b1_c1$ 被单独放入 inset，从而避免极小误差点把整张图的误差轴压扁。

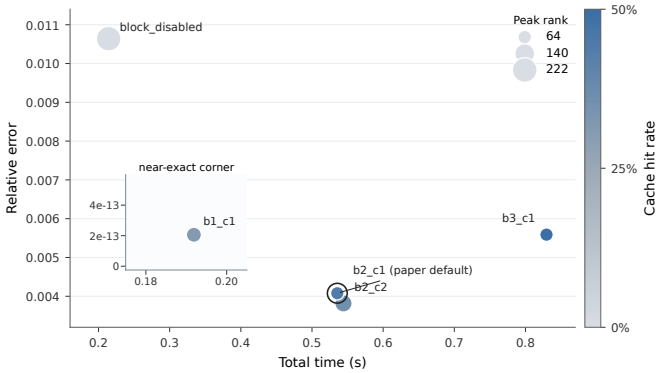

Figure 4: Grid 3×3 上的 blockwise 设计空间。主图聚焦 practical operating region，点面积编码 peak rank、颜色编码 cache hit rate；inset 单独展示 near-exact 的 $b1_c1$，从而使默认配置 $b2_c1$ 能以“平衡点”而非“被压扁的中间点”被读出。

在这样的视图下，$b2_c1$ 的角色更清楚了：它并不是误差最低、也不是时间最短，而是在当前候选项中提供了更均衡的工作点。具体而言，$b1_c1$ 虽然几乎达到精确误差，但总时间达到 2.3536 秒，明显慢于默认配置；$b2_c2$ 在误差上略优于默认配置（3.82×10^{-3} vs. 4.08×10^{-3}），但总时间增加到 0.5825 秒，且峰值秩升至 107.67；而 $b3_c1$ 则进一步牺牲精度换取另一种复用模式。因此，默认的 $b2_c1$ 并不是任意选取，而是在误差、时间、峰值秩与复用行为之间做出的平衡。

输出分块与缓存复用的精确数值。　Table 6 补充给出 blockwise / cache 组合的原始数值。在缓存复用方面，默认配置 $b2_c1$ 的缓存命中率为 0.440，缓存开启时总时间为 0.5342 秒，关闭缓存后增至 0.6566 秒，而误差保持不变。对 $b2_c2$ 也可观察到同样趋势：缓存开启时总时间为 0.5825 秒，关闭后为 0.6095 秒。这说明局部块键与状态缓存不仅在默认设置下有效，而且在相邻分块策略下也能带来一致的系统收益；正文中的 paired cache figure 则提供了更直观的视觉对照。

Table 5: Grid 3×3 上的内部机制证据。正文默认配置对应 `block_labels=2`、`chunk_size=1`、cache on、implicit sketch on。

Setting	rel. error	contract time (s)	total time (s)	$t_{\text{contract}}/t_{\text{total}}$	peak rank	# implicit sketches	cache hi
ASTNC default	4.08×10^{-3}	0.5340	0.5342	0.9997	64	13.67	0.44
no-blockwise	1.06×10^{-2}	0.5265	0.5265	0.9999	222	4.00	0
no-implicit	4.08×10^{-3}	1.1790	1.1792	0.9999	64	0	0.44
no-cache	4.08×10^{-3}	0.6561	0.6566	0.9992	64	13.67	0

Table 6: Grid 3×3 上的输出分块与缓存复用补充证据。$b2_c1$ 为正文默认配置。

Setting	rel. error	total time (s)	speedup vs. exact	cache hit rate	# cached states	mean rank	p
$b1_c1$	$<10^{-12}$	2.3536	20.07×	0.314	35	1.0	
$b2_c1$ (paper default)	4.08×10^{-3}	0.5342	88.45×	0.440	79	1.0	
$b2_c2$	3.82×10^{-3}	0.5825	81.12×	0.359	41	1.0	
$b3_c1$	5.59×10^{-3}	1.3057	36.19×	0.494	135	1.0	
$b2_c1$, cache off	4.08×10^{-3}	0.6566	71.96×	0	0	1.0	
$b2_c2$, cache off	3.82×10^{-3}	0.6095	77.51×	0	0	1.0	